Magpie — Sprint 7 Technical Spec (Phase 4 closer: eBay "Publish")

Audience: implementing agent (Codex) + Regan. **Builds on** Sprints 0–6 (`SPRINT_0_1_SPEC.md` … `SPRINT_6_SPEC.md`). **Status: planning / review — implementation starts only after (a) this spec is approved AND (b) Sprint 6 is formally closed under D-65 (production-connect checklist).**

> Build only what is in this document (D-65…D-75). **CI never calls eBay — fakes only, forever.** Real eBay operations happen exclusively in the manual first-flight protocol (§13), with Regan present. The publish gate (type-the-SKU) is the most important UI element in this sprint: it guards the first irreversible marketplace action in Magpie's life.

Conventions inherited: adapter ports + fakes in `integrations/ebay.py` (HTTP confined there); `apps/ebay` services + `apps/audit` rails from Sprint 6; encrypted fields via `EncryptedTextField` ; consolidated "PostgreSQL Validation" workflow; `manage.py backup` covers all new tables.

---

0. Locked decisions this spec implements

- **D-65** (supersedes D-62): sandbox = optional tool, never a gate. Sprint 6 closes on the production-connect checklist; Sprint 7 validates via the first-flight protocol (§13). CI fakes-only is unchanged and permanent.
- **D-66** Sprint 7 = Phase 4 closer in one sprint; 7a (staging) / 7b (publish) fault line pre-authorised **only** on Codex-reported size/instability evidence. Implementation gated on Sprint 6 closure.
- **D-67** Three new adapters + fakes: `EbayMediaAdapter` , `EbayInventoryAdapter` , `EbayTaxonomyAdapter` (client-credentials **app token**, strictly separate from the seller credential).
- **D-68** Staging pipeline: fresh EPS upload per staging → inventory item upsert → **unpublished** offer; withdrawable; one-time merchant location (address via env/UI only — never in code or seed).
- **D-69** Category = Taxonomy suggestions assist (production-only) + **manual ID**

entry always available and always wins; cache only selected category metadata/aspects with provenance (no tree mirroring). Condition = static per-profile map → `conditionId` + draft-level override.

- **D-70** Aspects pre-flight **blocks staging** on missing required aspects; explicit, audited Regan override; publish-error loop is the backstop; no auto-retry, no AI.
- **D-71** `ListingDraft.status` extends (+ `staged` / `published` / `publish_failed`); **one draft per item** may hold a live unresolved eBay state; all eBay state in `channel_data` (key list in §4.4); no `EbayOffer` model.
- **D-72** Publish gate: staged-review screen renders **server-side truth from `get_offer` + explicit warning + type-the-SKU confirmation (validated server-side)**; success → `listing_id` stored, item → listed; failure → back to `staged` with verbatim errors; withdraw only while unpublished; no live-listing revise/reprice/end in S7.
- **D-73** Closure = fakes-only CI green + the **first-flight protocol** executed with evidence. Fault attribution is strict: payload/auth/config/missing-policy errors are ours — no close until fixed; only documented eBay-infrastructure failures are external blockers warranting an adjudicated close.
- **D-74** CI guard inversion: the no-listing-write guard is **replaced** by the HTTP-confinement guard (all eBay network calls only in `backend/integrations/ebay.py`; `apps/ebay` has no network imports — `requests` / `httpx` / `aiohttp` / `urllib`; frontend never calls eBay; no eBay secrets in CI logs). Audit taxonomy extended (§4.6).
- **D-75** Out of scope (§12).

---

1. Repo additions

```
backend/integrations/ebay.py     # EXTEND: EbayMediaAdapter, EbayInventoryAdapter
                                 # EbayTaxonomyAdapter (+Http* and Fake* for each
backend/apps/ebay/
├── models.py                    # + EbayAppToken, MerchantLocation
├── staging.py                   # NEW — stage_draft / withdraw_staged
├── aspects.py                   # NEW — aspects pre-flight
├── publishing.py                # NEW — staged_review / publish_draft
├── management/commands/create_merchant_location.py
└── (serializers/views extended)
frontend: PublishPanel, CategoryPicker, AspectsChecklist, StagedReviewScreen,
          PublishConfirmDialog (type-the-SKU), WithdrawButton; listing/ebay API m
```

New migrations: `EbayAppToken`, `MerchantLocation` (+ backup coverage). **No new third-party dependencies.**

---

2. Configuration & scopes

No new env vars. **Build-start verification pin (D-60/D-67):** confirm `sell.inventory` covers Media API `createImageFromFile` + all Inventory API calls. If Media needs a distinct scope: add to `EBAY_SCOPES`, document that Regan must **reconnect once** (F-11), and record the finding in the close-out. Taxonomy uses the **basic `api_scope` on a client-credentials app token** (F-13) — never the seller token.

---

3. Models & adapters

3.1 `EbayAppToken`

`environment` (unique) · `access_token` (`EncryptedTextField`) · `expires_at`. Minted/refreshed on demand via client-credentials; **no seller identity, no refresh token**; deleting the seller credential must not affect it (and vice versa — test both).

3.2 `MerchantLocation`

`environment` (unique) · `merchant_location_key` · `name` · `country` · `postal_code` · `city` · `state` · `created_on_ebay` (bool) · `fetched_at`. Populated via the settings form or `create_merchant_location` command (values from input/env). **Never seeded, never hardcoded.** Calls `createInventoryLocation` once; audited (`ebay.location.created` / `ebay.location.refresh_failed`).

3.3 Adapter contracts (all raise `EbayUnavailable`; HTTP only in `integrations/ebay.py`)

```
class EbayMediaAdapter(Protocol):
    def upload_image(self, *, access_token: str, file) -> str: ...   # → EPS imag

class EbayInventoryAdapter(Protocol):
    def upsert_inventory_item(self, *, access_token, sku, payload) -> None: ...
    def create_offer(self, *, access_token, payload) -> str: ...     # → offer_id
    def update_offer(self, *, access_token, offer_id, payload) -> None: ...
    def withdraw_offer(self, *, access_token, offer_id) -> None: ...
    def publish_offer(self, *, access_token, offer_id) -> str: ...   # → listing_
```

```
    def get_offer(self, *, access_token, offer_id) -> dict: ...      # server-sid

class EbayTaxonomyAdapter(Protocol):                                  # app token
    def default_tree_id(self, *, marketplace: str) -> str: ...
    def suggest_categories(self, *, q: str) -> list[dict]: ...        # PRODUCTIO
    def item_aspects(self, *, category_id: str) -> list[dict]: ...    # name, req
```

Fakes: deterministic. `FakeMedia` → `fake-eps://{n}.jpg`; `FakeInventory` tracks offers in memory with publish success/failure toggles + canned eBay-style error payloads; `FakeTaxonomy` → canned suggestions + an aspects fixture containing required aspects (so the pre-flight block path is testable). Fake `suggest_categories` in sandbox-mode mimics the real unsupported response so the UI fallback is exercised.

---

4. Staging (`staging.py`)

4.1 `stage_draft(draft, *, override_missing_aspects=False)`

Ordered, audited, idempotent (re-staging replaces):

1. **Guards:** draft not `published`; item not sold/archived; no *other* draft on this item in `staged/published/publish_failed` (D-71); ≥1 photo (fail on zero); ≤24 photos (defensive); `category_id`, `condition_id`, all three policy IDs, and merchant location present; **aspects pre-flight passes or `override_missing_aspects=True`** (override audited with the missing list in the payload).
2. **Fresh EPS upload** of the draft's ordered **processed** photos → `channel_data.eps_image_urls` (no reuse across stagings — D-68/F-12 purge-proofing). Audit `ebay.media.uploaded` (count, not URLs… URLs are fine — no secrets; include count + first URL).
3. `upsert_inventory_item` (SKU = item SKU; title, description, aspects from specifics, condition, imageUrls, quantity). Audit `ebay.inventory_item.upserted`.
4. `create_offer` (or `update_offer` on re-stage) — EBAY_AU, format, price/currency, `category_id`, policy IDs, `merchant_location_key`. **Offer is UNPUBLISHED (F-05).** Audit `ebay.offer.created` / `updated`.
5. Write `channel_data`: `offer_id`, `inventory_item_sku`, `staged_at`, `last_payload_snapshot`; status → `staged`. Any step failure → status unchanged (or stays `staged` on re-stage failure), `channel_data.last_ebay_error` = verbatim error, surfaced in UI.

4.2 `withdraw_staged(draft)`

Only for `staged` / `publish_failed` with an unpublished offer: `withdraw_offer` → clear `offer_id` / `staged_at` (snapshots retained) → status → `ready` → audit `ebay.offer.withdrawn`.

4.3 Category & condition

`CategoryPicker`: search box → `suggest_categories` (production; on sandbox/fake-unsupported, the UI shows "suggestions unavailable — enter category ID" — manual field is **always** visible and **always wins**). Chosen `{category_id, category_tree_id, name}` → `channel_data` with `fetched_at`. Condition: static `CONDITION_MAP` per profile (e.g. NEW→1000, USED→3000; collectibles-grade nuances documented in the map) + `channel_data.condition_id` override field.

4.4 `channel_data` keys (D-71 — the complete set)

`category_id, category_tree_id, category_name, condition_id, merchant_location_key, payment_policy_id, fulfillment_policy_id, return_policy_id, eps_image_urls, inventory_item_sku, offer_id, listing_id, staged_at, published_at, last_ebay_error, last_payload_snapshot, last_offer_snapshot`. No over-normalising; no new models for these.

4.5 Aspects pre-flight (`aspects.py`)

`check_aspects(draft)` → `{satisfied_required, missing_required, optional_known, unmapped_specifics}` by comparing `item_aspects(category_id)` against the draft's `item_specifics`. `missing_required` non-empty ⇒ staging **blocked** (D-70) unless overridden. Fetched aspects cached per category with `fetched_at` (provenance; no bulk tree fetch). Audit `ebay.taxonomy.aspects_fetched` (and `ebay.taxonomy.category_suggested` on suggestion use).

4.6 Audit actions added (D-74)

`ebay.media.uploaded` · `ebay.inventory_item.upserted` · `ebay.offer.created` · `ebay.offer.updated` · `ebay.offer.withdrawn` · `ebay.publish.attempted` · `ebay.publish.succeeded` · `ebay.publish.failed` · `ebay.taxonomy.category_suggested` · `ebay.taxonomy.aspects_fetched` · `ebay.location.created` · `ebay.location.refresh_failed` — payloads secret-free as ever.

---

5. Publishing (`publishing.py`)

- **`staged_review(draft)` → calls `get_offer` and returns eBay's record (not Magpie's intent): offer ID, SKU, title, category ID/name, condition, price, quantity, format, policy IDs, location key, photo count + current aspect warnings. Stored as `channel_data.last_offer_snapshot`.**
- **`publish_draft(draft, *, confirm_sku: str)` → server-side check `confirm_sku == item.sku` (the UI dialog is not the enforcement point); status must be `staged`; audit `ebay.publish.attempted` → `publish_offer` → on success: `channel_data.listing_id` + `published_at`, status → `published`, item status → listed, audit `ebay.publish.succeeded`; on failure: status → `publish_failed`, verbatim errors → `last_ebay_error`, audit `ebay.publish.failed`. `publish_failed` drafts can be edited → re-staged → re-published; no auto-retry.**
- **Once `published`: no revise/reprice/end in Magpie (D-75) — Seller Hub manages the live listing; Magpie displays `listing_id` with a link.**

---

6. API endpoints

Method	Path	Purpose
POST	`/api/listing-drafts/{id}/stage/`	Body `{override_missing_aspects?: bool}`; runs §4.1.
POST	`/api/listing-drafts/{id}/withdraw/`	§4.2.
GET	`/api/listing-drafts/{id}/aspects-check/`	Pre-flight result for the UI.
GET	`/api/listing-drafts/{id}/staged-review/`	Server-side truth via `get_offer`.
POST	`/api/listing-drafts/{id}/publish/`	Body `{confirm_sku}`; §5.
GET	`/api/ebay/category-suggestions/?q=`	Production-assist; clean "unsupported" response otherwise.
GET	`/api/ebay/category-aspects/?category_id=`	Cached aspects.
GET/POST	`/api/ebay/merchant-`	View / one-time create (form values,

	`location/`	never stored in code).

---

7. Frontend — the Publish panel

On the draft editor (post-Sprint-5 Listing panel): **CategoryPicker** (suggestions when available + always-visible manual ID field), **AspectsChecklist** (✓ satisfied / ✕ missing-required / optional / unmapped; Stage disabled while ✕ exist, with an explicit "Override and stage anyway" requiring a checkbox + reason), **Stage** button → staged-state card (offer ID, staged_at, Withdraw), **Review & Publish** → `StagedReviewScreen` (eBay's record + "**This will create a live eBay listing**" warning) → `PublishConfirmDialog` with a **type-the-SKU input** (button disabled until exact match; server re-validates) → published card (listing ID + eBay link) or verbatim-error display on failure. SANDBOX/PRODUCTION badge carried over everywhere eBay state is shown.

---

8. CI changes (D-74)

In `postgresql-validation.yml` : **delete** the no-listing-write guard (the v2.7 breadcrumb, executed consciously); **add** the confinement guard — fail if `requests|httpx|aiohttp|urllib` is imported anywhere under `backend/apps/` (allowed only in `backend/integrations/`), and fail if eBay hostnames appear outside `integrations/` . CI env never contains real eBay secrets; fakes only.

---

9. Acceptance criteria

1. Migrations (`EbayAppToken` , `MerchantLocation`) apply on CI; backup/restore covers them.
2. **Staging end-to-end against fakes**: guards (zero-photo fail, >24 fail, one-staged-draft-per-item, missing category/condition/policies/location fail with clear messages), fresh-upload-per-staging proven (fake URL counters), `channel_data` fully populated, full audit sequence emitted.
3. **Pre-flight blocks**: missing required aspects → staging rejected; override path works and is audited with the missing list; satisfied case passes.
4. **Publish gate**: wrong/absent `confirm_sku` rejected **server-side**; publish only from `staged` ; success path sets `listing_id` + flips item to listed; failure path →

`publish_failed` with verbatim errors; re-stage after failure works; withdraw only while unpublished.

5. Taxonomy: suggestions return cleanly in production-mode fakes, "unsupported" in sandbox-mode fakes, and the UI manual path works in both; aspects cached with provenance; **app token fully independent of the seller credential** (delete either, the other still works).
6. Guard inversion in the workflow file (old guard gone; confinement guard active and passing); no eBay secret material in CI logs.
7. Frontend: AspectsChecklist states, Stage disable/override, type-the-SKU enablement, staged review renders `get_offer` data, verbatim error display; typecheck + build green.
8. "PostgreSQL Validation" green (fakes only). **Closure additionally requires §13 executed with evidence** (`FIRST_FLIGHT_EVIDENCE.md`), under D-73 fault attribution.

---

10. Test plan

Backend: every §9 item as unit/integration tests against fakes — staging guards individually; idempotent re-stage (update_offer path); withdraw transitions; aspects pre-flight (block/override/satisfied + cache provenance); publish SKU validation (server-side), success/failure transitions, item flip only on success; one-staged-draft constraint; app-token mint/refresh/independence both directions; merchant-location command + form (no address in fixtures — use obviously fake values); audit actions + secret-free payloads; backup/restore. **Frontend (light):** picker fallback, checklist rendering, override checkbox flow, SKU-match button enablement, error display. **CI:** confinement guard self-test (a deliberate bad import in a scratch branch fails it — verified once during review, not committed).

---

11. Seed

No eBay state seeded. No merchant location, no app token, no category data. Fakes provide everything tests need via fixtures.

12. Out of scope (D-75)

Order/sales sync · `SaleRecord` · realized-profit dashboards · revise/reprice/relist/end-live-listing automation · auction scheduling · batch publish · multi-variation listings ·

Promoted Listings · second marketplaces · Browse/Buy APIs · AI anything.

---

13. First-flight protocol (closure evidence — Regan present throughout)

Prerequisites: Sprint 6 closed under D-65 (production connected); `FeeSchedule` real rates entered; merchant location created; **first-flight item nominated** (real, cheap, expendable, photographed, valued, clean draft); Seller Hub end-listing path located in advance.

1. Fakes-only CI green. 2. Production connect verified (status endpoint). 3. Draft the first-flight item; pick category (suggestions or manual); pre-flight green. 4. **Stage** in production — media upload, inventory item, **unpublished** offer; verify in audit + `channel_data`. 5. Open **staged review** — confirm eBay's record matches intent. 6. **Rehearse withdrawal**: withdraw the staged offer, verify cleared, then re-stage. 7. **Publish**: type-the-SKU confirm, Regan present → `publishOffer`. 8. Verify the live listing on eBay; `listing_id` stored; item status listed; full audit trail; **no secrets anywhere**. Capture all of it in `FIRST_FLIGHT_EVIDENCE.md`. If any step fails: D-73 fault attribution decides — our error = fix before close; documented eBay-infrastructure failure = Regan adjudicates.

Hand-off notes for the implementing agent

- **The server-side SKU check is the publish gate.** The dialog is UX; the check in `publish_draft` is the law. Do not weaken it, do not add a bypass flag.
- Staging must be boringly reversible. If withdraw doesn't cleanly return a draft to `ready`, the sprint isn't done.
- The pre-flight override is an audited escape hatch, not a default — never pre-check the override box.
- Replace the CI guard consciously (D-74); deleting it without adding the confinement guard is a defect.
- If you find yourself writing revise/end-listing code for live listings, you have left the sprint.
- When ambiguous, prefer the smaller implementation that satisfies §9.